

第 32 章 容器持久化、用户 systemd 与 Quadlet

把一次性运行实例转化为可重建声明，并证明它在用户未登录时仍能启动、保存数据和提供功能。

对象模型 操作语义 验证 诊断 经典任务

大字号阅读版

本章阅读导航

先抓住一条主线：容器实例只是“现在发生了什么”，Quadlet 才描述“下一次应该怎样重建”；用户 systemd manager、**[Install]** 与 linger 共同决定它何时被启动。

01 识别作用域确认目标用户、rootless Podman 与 user manager

02 记录当前实例保留镜像、名称、端口、环境、挂载和数据证据

03 写入声明把期望状态翻译为 `.container` 字段

04 生成与启动 `daemon-reload` 生成 unit，start/restart 应用配置

05 分层验证 unit、容器、数据和协议功能分别取证

06 验证生命周期注销和冷启动后，先在目标用户未登录时测功能

专题地图 - **知识专题** 当前实例与声明配置 - **知识专题** 用户 systemd manager 与 linger - **知识专题** Quadlet 搜索路径、名称映射与生成物 - **操作专题** 把 `podman run` 参数翻译为 `.container` - **操作专题** 从声明到运行：reload、start 与证据链 - **知识专题** 依赖、顺序与 Restart - **操作专题** 数据持久化、声明变更与镜像更新 - **诊断专题** unit 未生成、启动失败、循环重启与冷启动失败 - **经典任务** 迁移现有 rootless 容器并完成未登录验收

阅读时持续回答 1. 当前看到的是运行实例、源声明，还是 generated unit？ 2. 命令连接的是系统 manager，还是目标用户的 manager？ 3. `Linger=yes` 证明了什么，又没有证明什么？ 4. 修改 `.container` 后，何时需要 reload，何时需要 restart？ 5. `active`、容器运行、端口映射和业务功能分别属于哪一层？ 6. 数据是否真正脱离容器可写层？ 7. 冷启动验证是否在目标用户首次登录之前完成？

章节边界：容器基本运行归第 31 章；systemd 通用模型归第 12 章；SELinux 持久规则归第 29 章。本章只讲把这些前置对象组织为可重建的用户级容器服务。

第 32 章 · 正文

容器能够运行，不等于它已经成为一项可维护的系统服务。手工执行一次 `podman run`，只建立了一个当前实例；用户退出、主机重启、镜像更新或配置变化以后，这个实例是否还能以相同参数恢复，取决于另一个层次的声明和生命周期管理。本章把容器从“当前正在运行的对象”提升为“可以由用户 systemd manager 重建和监督的声明对象”。最常见的误判是把 `Linger=yes`、`active` 或 `podman ps` 中的一行运行状态直接扩大解释为“重启后必然可用”。正确路径必须把源声明、生成 unit、运行实例、外部数据、功能和登录外生命周期分开取证。主线不是把长串 `podman run` 塞进手写 service，也不是把旧的 `podman generate systemd` 当成唯一答案，而是使用 Quadlet `.container` 文件表达期望状态，再由 generator 生成普通 `.service`，最后通过 `systemctl --user`、`loginctl`、`journalctl` 和 Podman 证据完成分层验收。

概念

声明式容器配置描述的是“下一次应当怎样创建容器”，而不是当前实例已经发生的全部历史。当前容器可以通过 `inspect` 观察，Quadlet 则是后续重建的配置真源；两者发生漂移时，必须先保存当前事实，再决定哪些参数进入长期声明。

概念

用户 `systemd manager` 属于一个具体 UID，并通过 `systemctl --user` 管理该用户的 unit。它与 PID 1 的 `system scope` 是两个管理域，所以 `root` 的普通 `systemctl`、`root` 的 Podman 存储和目标普通用户的 `rootless` 容器不能互相替代。

概念

`linger` 是用户登录外生命周期策略。启用后，系统可在开机时创建该用户的 `manager`，并在最后一次注销后保留它；`linger` 只回答 `manager` 是否能脱离登录会话存在，不能证明 Quadlet 路径、字段、数据权限或应用功能正确。

概念

Quadlet 使用 `.container` 等声明文件描述 Podman 对象。`generator` 在 `user manager` 启动或 `daemon-reload` 时读取源文件并生成普通 `service`，使管理员维护简短字段，而不是长期维护一份由旧版本 Podman 展开的复杂 `ExecStart`。

概念

生成 `unit` 是 Quadlet 的派生结果。`report.container` 通常生成 `report.service`；它可以被 `status`、`show` 和 `journal` 观察，但不应直接编辑。配置变化应回到源声明，再通过 `reload` 重新生成。

概念

容器重建意味着旧实例可以被停止、删除并按声明重新创建，因此容器 ID 变化是正常现象。真正需要保持的是题目指定的名称、端口、环境、挂载和功能终态，而不是让某个实例永远不变。

概念

持久数据必须位于宿主目录或 Podman volume 等独立数据层。只在容器可写层中的文件会随实例删除而失去；最有区分度的验证不是简单 `restart`，而是删除旧实例、由 Quadlet 重建后，新实例仍能读取同一份外部数据。

版本敏感：先确认 Podman，再选择主线

Quadlet 从 Podman 4.6 开始提供，而 RHEL 9 的不同小版本可能携带不同 Podman 版本。进入操作前先执行 `podman version` 并查看本机 `man podman-systemd.unit`。本章以 Quadlet 为主线；较早环境中的 `podman generate systemd` 只作为兼容背景，不作为唯一答案。

操作语义速查

`systemctl --user`

SYNOPSIS

```
systemctl --user COMMAND [UNIT ...]
```

连接当前用户的 systemd manager，观察或改变 user unit。对 rootless Quadlet，必须在目标用户的完整会话中执行，不能把 system scope 的结果混进来。

`status NAME.service`

查看当前加载和运行摘要；适合快速定位，但不能替代结构化字段与功能测试。

`show NAME.service -p ...`

读取 `LoadState`、`ActiveState`、`SubState`、`Result`、`NRestarts` 等最小状态向量。

`start / stop / restart`

改变当前运行状态；`restart` 才会让运行实例采用已重新生成的配置。

`loginctl enable-linger`

SYNOPSIS

```
loginctl enable-linger USER
loginctl show-user USER -p Linger -p State
```

让指定用户的 manager 可以在开机时创建，并在最后一次注销后继续存在。该操作通常需要管理员授权。

`enable-linger USER`

写入持久的 linger 状态；它不创建或修复容器声明。

`show-user USER -p Linger`

验证策略状态；`Linger=yes` 仍不能证明 unit 或应用已经正常。

`user-status USER`

观察用户 manager、会话和相关进程；适合冷启动后由其他管理员账号取证。

`Quadlet .container`

SYNOPSIS

```
~/ .config/containers/systemd/NAME.container
```

```
[Container]
Image=IMAGE
ContainerName=NAME
PublishPort=HOST:CONTAINER
Volume=SOURCE:TARGET[:OPTIONS]
Environment=KEY=VALUE
```

把容器的期望状态写成字段。文件名决定生成的 service 名，字段决定下一次创建的实例参数。

Image=

必需字段；优先使用完整镜像引用，并区分可变 tag 与实际 image ID。

ContainerName=

覆盖默认的 `systemd-NAME` 容器名；显式命名时要先处理同名旧实例。

PublishPort=

左侧宿主端口，右侧容器端口；端口映射存在不等于协议功能正确。

Volume=

左侧外部数据源，右侧容器内目标；需要同时满足传统权限、UID 映射和 SELinux 边界。

Environment=

可重复使用；这里遵循 unit/Quadlet 语法，不能机械套用 shell 引号规则。

daemon-reload 与声明应用

SYNOPSIS

```
systemctl --user daemon-reload
systemctl --user restart NAME.service
```

reload 让 manager 重新运行 generator 并读取源声明，restart 才让当前运行实例采用新生成的 service。两步改变的对象不同。

daemon-reload

证明 manager 已重新读取配置入口；它不会自动重启业务。

restart NAME.service

停止旧实例并按新配置重建；执行前应确认持久数据和可接受的停机影响。

[Install] WantedBy=default.target

表达 user manager 内的启动关系；generated service 不能通过普通 `enable` 流程获得持久启用关系。

journalctl --user-unit

SYNOPSIS

```
journalctl --user-unit=NAME.service [-b] [-n N] [--no-pager]
```

读取 user unit 的生成、启动、退出和重启证据。查询必须处于正确用户作用域，日志是进入失败分支的入口，而不是事后装饰。

`-b`

限制为当前 boot，便于区分历史故障和本次冷启动。

`-n N`

先读取最近 N 条，快速找到第一条高价值错误。

`--since`

按变更时间缩小范围，避免在大量历史记录中猜测。

重启后未登录验证

SYNOPSIS

```
# 主机重启后，先不要登录目标用户
curl -fsS http://SERVER:PORT/
loginctl user-status USER

# 随后进入目标用户完整会话复核
systemctl --user status NAME.service
journalctl --user-unit=NAME.service -b
```

冷启动验收必须避免登录动作污染证据。先从其他账号或远端验证业务功能，再进入目标用户会话查看 unit、容器和日志。

外部协议测试

证明真实入口在目标用户尚未登录时已经可用；比单看 `active` 更接近终态。

`loginctl user-status USER`

观察 user manager 是否已经存在；它仍不能替代协议和数据验证。

登录后回查

用于解释启动链和日志，不得把登录后才恢复的服务误判为开机成功。

[知识专题] 当前实例与声明配置：先分清“已经发生”与“下一次应发生”

把现有容器迁移为 systemd 服务时，最危险的做法是直接抄一条记忆中的模板。现有实例可能包含题目要求保留的数据、端口、环境变量和用户映射，也可能包含临时试验参数。正确切入点是先建立当前事实，再决定哪些事实应进入声明，哪些应该被纠正。

① [查询] 用当前实例建立迁移基线

`podman ps -a` 能显示名称、状态、端口等摘要，`podman inspect` 能读取创建时的详细配置。它们回答的是：当前这个实例怎样被创建、现在处于什么状态。容器 ID 不是长期配置标识；实例被删除和重建后，ID 通常变化。

建议在迁移前至少记录：

```
podman ps -a --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'\npodman inspect report > ~/report-before.json\npodman port report\npodman logs --tail 50 report
```

`inspect` 是基线证据，不是自动生成声明的权威模板。它可能保留历史临时选项，仍需根据任务目标筛选。

② [知识点] 声明配置描述下一次生成的期望状态

`.container` 文件表达下一次服务启动时应使用的镜像、名称、端口、挂载和环境。修改声明不会在同一瞬间改写当前容器；`systemd` 先要重新加载生成结果，运行实例再通过 `restart` 或下一次启动采用新配置。

因此需要区分三次变化：

```
编辑 .container\n→ systemctl --user daemon-reload 重新生成 service\n→ systemctl --user restart NAME.service 重建运行实例
```

只完成第一步时，当前业务可能仍在旧参数；只执行 `restart` 而没有 `reload`，也可能继续使用旧的 `generated unit`。

③ [操作决策] 从现有事实筛选可维护终态

迁移时将现有事实分成三类：

类别	例子	处理
必须保留	业务数据目录、规定端口、必要环境变量	进入声明并逐项验证
应纠正	临时随机端口、错误名称、数据只在可写层	按任务终态重构
暂不进入主线	调试用 <code>shell</code> 、临时 <code>capability</code> 、实验标签	有明确需求才保留

迁移前不要先删除旧实例。先确认数据在何处、是否有外部副本、端口是否被依赖，再安排最小停机切换。

Cheatsheet

``inspect`` 建立当前事实；``container`` 表达下一次期望；编辑、`reload`、`restart` 是三个不同动作；容器 ID 变化不等于数据丢失，数据位置才是关键。

[知识专题] 用户 systemd manager：作用域比命令名字更重要

容器由哪个用户运行，会决定 Podman 存储、容器可见性、用户 `bus`、`unit` 搜索范围和日志作用域。很多“`unit not found`”并不是 `unit` 文件不存在，而是当前命令连到了错误的 `manager`。

① [知识点] system scope 与 user scope 是两个管理域

普通命令：

```
systemctl status sshd.service
```

连接系统 manager，管理系统级 unit。用户命令：

```
systemctl --user status report.service
```

连接当前用户的 manager，管理该用户的 unit。对于 rootless 容器，Quadlet、Podman 存储和 user unit 必须归属于同一目标用户。

以下证据应在目标用户上下文中取得：

```
id
podman ps -a
systemctl --user list-unit-files
systemctl --user status report.service
```

不要把 root 的 `podman ps` 为空解释为目标用户没有容器，也不要将系统 manager 的 `Unit report.service could not be found` 直接解释为 Quadlet 未生成。

② [知识点] 登录会话与用户 manager 相关，但不是同一对象

用户通过 SSH、控制台或其他 PAM 会话登录时，通常会得到用户 runtime 目录和 user manager 访问环境。注销最后一个会话后，如果没有 linger，用户 manager 可以被停止；由它监督的长期 user service 也会失去管理者。

查询用户生命周期：

```
loginctl show-user appsvc -p Linger -p State -p Sessions
loginctl user-status appsvc
```

`State=active` 只表示当前存在活跃会话或 manager 状态，不等同于重启后会自动创建；`Linger=yes` 才是脱离登录会话的持久策略之一。

③ [边界] linger 只解决 manager 的登录外生命周期

启用：

```
sudo loginctl enable-linger appsvc
```

验证：


```
logindctl show-user appsvc -p Linger
```

`Linger=yes` 可以支持开机时创建 user manager，并使最后一次注销后 manager 继续存在。但它不能证明：

- `.container` 在正确目录；
- `[Install]` 已表达启动关系；
- generated service 能加载；
- 容器镜像可用；
- 端口和数据目录无冲突；
- 应用功能正确。

Cheatsheet

rootless 容器、Quadlet、user unit 和日志都要在同一 UID 作用域内取证；`--user` 不是装饰选项；linger 解决登录外生命周期，不解决配置正确性。

[知识专题] Quadlet 的输入、搜索路径和生成物

Quadlet 的价值在于让管理员维护简短的容器声明，由 Podman generator 根据当前版本生成具体 `ExecStart=` 等实现细节。这样升级 Podman 后不需要继续维护一份历史生成脚本。

① [查询] 先按运行身份确定 Quadlet 搜索路径

本章主线使用当前用户目录：

```
$XDG_CONFIG_HOME/containers/systemd/  
~/.config/containers/systemd/
```

管理员也可以为特定 UID 或全部用户提供系统级用户声明：

```
/etc/containers/systemd/users/<UID>/  
/etc/containers/systemd/users/
```

rootful 系统声明使用：

```
/etc/containers/systemd/  
/usr/share/containers/systemd/
```

考试任务应先明确 rootless 还是 rootful。把 rootless Quadlet 放入 `~/.config/systemd/user/` 是常见路径错误；该目录适合普通 user unit，不是本章 `.container` 主路径。

② [知识点] 文件名映射为 service 和默认容器名

`report.container` 生成 `report.service`。若未写 `ContainerName=`，Podman 默认采用带 `systemd-` 前缀的名称，例如 `systemd-report`，以减少与手工容器重名。

```
report.container
  ↓ generator
report.service
  ↓ podman run
systemd-report    (未显式 ContainerName 时)
```

任务明确要求容器名时应写 `ContainerName=report`，并在迁移前处理已有同名实例，避免启动时发生 `name already in use`。

③ [边界] generated service 是派生状态，不是配置真源

generator 在开机和 manager reload 时读取声明。管理员应修改 `.container`，然后执行：

```
systemctl --user daemon-reload
```

可以使用以下命令观察生成结果：

```
systemctl --user status report.service
systemctl --user cat report.service
systemctl --user show report.service -p FragmentPath -p SourcePath -p LoadState
```

实际字段显示会随 systemd/Podman 版本变化，不能把某条固定输出当成唯一评分证据。核心判断是 service 能否被正确加载，并且来源可追溯到目标 `.container`。

Cheatsheet

rootless 路径是 `~/ .config/containers/systemd/``；后缀必须是 ``container``；``NAME.container`` → ``NAME.service``；默认容器名可能是 ``systemd-NAME``；只编辑源声明，不编辑生成物。

[操作专题] 把 `podman run` 参数翻译为 `.container`

Quadlet 不是另一种 shell 命令拼接，而是把容器参数映射为可读字段。转换时应逐项回答：该字段控制哪个对象，是否能从题目终态验证，是否需要重复出现。

① [操作] 镜像与容器名称

作用对象：下一次创建的容器实例。

基本形式：

```
[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
```

`Image=` 是 `.container` 的必需字段。优先使用完整镜像名，避免短名称解析不确定。`tag` 表示可变名称；若任务要求精确不可漂移版本，可使用 `digest`，但考试题通常会给定 `tag` 或完整引用。

`ContainerName=` 是可选字段。省略时要接受默认 `systemd-` 前缀名称；题目指定名字时再显式设置。

验证：

```
podman inspect report --format '{{.Name}} {{.ImageName}} {{.Image}}'
```

不要把 `podman images` 中存在目标 `tag` 当成运行实例已经采用该 `image ID` 的证据。

② [操作] 端口与环境变量

基本形式：

```
[Container]
PublishPort=18080:8080
Environment=REPORT_MODE=exam
Environment=LOG_LEVEL=info
```

`PublishPort=` 对应 `podman run --publish`，可重复使用。`18080:8080` 表示宿主端口 18080 映射到容器端口 8080。只有容器端口而没有固定宿主端口时可能产生动态宿主端口，不适合要求固定入口的题目。

`Environment=` 对应 `--env`，可重复使用。包含空格或特殊字符时要遵循 `systemd` 单元语法，避免随意套用 `shell` 引号规则。

验证：

```
podman port report
podman inspect report --format '{{json .Config.Env}}'
curl -fsS http://127.0.0.1:18080/
```

端口存在只能证明转发表达，`curl` 或实际协议测试才进入功能层。

③ [操作] 数据卷、服务重启与开机关系

基本形式：

```
[Container]
Volume=/home/appsvc/report-data:/var/lib/report:Z

[Service]
Restart=on-failure
RestartSec=5

[Install]
WantedBy=default.target
```

`Volume=` 对应 `--volume`，可重复使用。宿主路径在左，容器路径在右。`:Z` 为该容器准备私有 SELinux 标签；共享语义和完整标签诊断归第 29、31 章，本章只要求保留最小安全边界。

`Restart=on-failure` 属于生成 service 的 `[Service]` 配置，适合进程异常退出后重试；它不会修复不可拉取镜像、重名容器、端口占用和持续权限错误，只会让失败反复出现。

Quadlet 生成的 service 属于 `transient/generated unit`，不能通过 `systemctl --user enable report.service` 获得持久启用关系。自动启动关系必须写进源文件 `[Install]`，由 generator 在生成阶段应用；用户级服务通常使用 `default.target`。

完整示例：

```
[Unit]
Description=Persistent rootless report container

[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
PublishPort=18080:8080
Volume=/home/appsvc/report-data:/var/lib/report:Z
Environment=REPORT_MODE=exam

[Service]
Restart=on-failure
RestartSec=5
TimeoutStartSec=900

[Install]
WantedBy=default.target
```

`TimeoutStartSec=900` 不是所有题目都需要。镜像首次拉取可能超过 `systemd` 默认启动超时，只有证据表明启动卡在拉取且网络正常时，才考虑预拉镜像或延长超时。

Cheatsheet

``Image`` 必需；``ContainerName`` 控制实例名；``PublishPort`` 左宿主右容器；``Volume`` 左源右目标；``Environment`` 可重复；``Restart`` 不修复根因；Quadlet 开机关系写 ``[Install]``。

[操作专题] 从声明到运行：reload、start 与证据链

最小流程不是一句 `start`。每一步产生不同证据，失败时应停在当前层调查，而不是继续堆命令。

① [操作] 建立目录和声明

```
install -d -m 0755 ~/.config/containers/systemd
vi ~/.config/containers/systemd/report.container
```

检查：

```
ls -l ~/.config/containers/systemd/report.container
sed -n '1,200p' ~/.config/containers/systemd/report.container
```

文件应属于目标用户，宿主数据目录也要在该用户和容器映射可访问的权限范围内。不要为了跳过调查直接 `chmod 777`。

② [操作] 重新生成并启动 unit

```
systemctl --user daemon-reload
systemctl --user start report.service
```

若需要立即应用声明变更：

```
systemctl --user restart report.service
```

`daemon-reload` 让 manager 重新读取 source unit 并运行 generator，不会自行重启已运行服务。`restart` 负责让新生成的 service 进入运行实例。

③ [验证] 沿 manager、实例和功能三层取证

```
systemctl --user status report.service --no-pager
systemctl --user show report.service \
  -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
podman ps -a --filter name=report
podman port report
curl -fsS http://127.0.0.1:18080/
```

`ActiveState=active` 证明 systemd 当前认为主服务活动；它不能独立证明应用内容正确。若镜像内进程保持运行但返回错误页面，unit 仍可能 active。

帮助入口：

```
man podman-systemd.unit
man systemctl
man systemd.service
```

Cheatsheet

创建声明后 reload；需要当前采用新配置再 restart；`status/show` 查 manager，`podman` 查实例，协议请求查功能。

[知识专题] 依赖、顺序与 Restart：三种不同关系

容器服务可能依赖另一个服务、挂载点或网络准备。依赖关系不能只靠排列文件名，也不能只写 `After=` 就假设依赖对象一定会启动。

① [知识点] `Requires=` 表达启动和失败关系

```
[Unit]
Requires=database.service
```

要求 database unit 一起参与事务；所需 unit 无法启动时，当前 unit 通常不能成功进入目标状态。它表达“需要谁”，不保证先后顺序。

② [知识点] `After=` 只表达顺序

```
[Unit]
After=database.service
```

表示两者都进入同一事务时，本 unit 在 database 之后启动。它不会主动把 database 拉入事务。常见组合是：

```
[Unit]
Requires=database.service
After=database.service
```

依赖其他 Quadlet 时使用生成后的 `.service` 名称，例如 `After=database.service`，不是 `After=database.container`。

③ [边界] Restart 处理退出结果，不处理声明错误

```
[Service]
Restart=on-failure
RestartSec=5
```

该策略处理非零退出、信号终止等失败。以下问题通常会形成稳定失败或循环：

- `ContainerName=` 与旧实例重名；
- 宿主端口已占用；
- 数据目录不可访问；
- 应用参数错误，启动即退出；
- 镜像引用无效。

出现循环时先取证：

```
systemctl --user show report.service -p Result -p NRestarts
journalctl --user-unit=report.service -b -n 100 --no-pager
podman ps -a --filter name=report
```

不要把 `Restart=always` 当成“提高可靠性”的无条件答案。若应用正常完成后本应退出，`always` 会制造无意义循环。

Cheatsheet

`Requires` 解决“需要谁”，`After` 解决“谁先谁后”，`Restart` 解决“失败退出后怎么办”；三者不可互换。

[操作专题] 让数据独立于容器实例

持久化的目标不是让容器 ID 永远不变，而是让业务数据不依赖一次实例的可写层。最强的验证不是“容器重启后文件还在”，而是删除并由声明重建实例后，外部数据仍能被新实例读取。

① [查询] 识别数据究竟位于哪一层

调查现有挂载：

```
podman inspect report --format '{{json .Mounts}}'
podman volume ls
findmnt -T /home/appsvc/report-data
```

如果关键文件只存在容器内且没有对应挂载，应先导出或复制到规划的持久目录，再删除旧实例。不要先 `podman rm` 再确认数据。

② [操作] 准备宿主目录和最小权限

```
install -d -m 0750 ~/report-data
printf 'persistent-marker\n' > ~/report-data/marker.txt
ls -ldZ ~/report-data
```

宿主传统权限、容器内 UID、rootless 用户映射和 SELinux 标签共同决定访问。默认排障顺序：

- 宿主路径是否正确
 - 所有者和模式是否允许
 - 容器内进程使用哪个 UID
 - 当前 SELinux 上下文和 AVC

禁止以 `chmod 777` 或关闭 SELinux 代替调查。

③ [验证] 用删除和重建证明数据独立

最低层：

```
podman exec report cat /var/lib/report/marker.txt
```

更强验证：

```
systemctl --user stop report.service
podman ps -a --filter name=report
systemctl --user start report.service
podman exec report cat /var/lib/report/marker.txt
```

若维护窗口和任务允许，可在确认数据外置后删除实例，再由 unit 重建；不要删除宿主数据目录或 named volume。

```
systemctl --user stop report.service
podman rm report
systemctl --user start report.service
podman exec report cat /var/lib/report/marker.txt
```

该流程证明数据跨实例，而不是只跨进程 restart。实际考试中是否手工 `podman rm` 要依据题目和现有状态决定，不把破坏性步骤当成无条件模板。

Cheatsheet

可写层随实例；外部目录/volume 才是持久层；先定位数据再删除；最强验证是新实例读取旧数据。

[操作专题] 声明变更、镜像更新与重建边界

容器维护常见误判是“镜像已经 pull，所以服务已经更新”。本地 tag 指向新 image ID，不会自动把已运行容器替换成新实例；Quadlet 源文件变化也不会自动让当前容器采用新值。

① [查询] 变更前保存实例和功能基线

```
podman inspect report > ~/report-before.json
systemctl --user show report.service \
  -p ActiveState -p SubState -p Result -p NRestarts
curl -fsS http://127.0.0.1:18080/ > ~/report-before.out
```

基线至少覆盖运行状态、镜像、端口、挂载、环境和业务输出。

② [操作] 修改声明后 reload + restart

```
vi ~/.config/containers/systemd/report.container
systemctl --user daemon-reload
systemctl --user restart report.service
```

若只改了宿主数据内容或应用自身可热加载配置，是否需要容器 restart 取决于应用；不要把 `systemd daemon-reload` 误认为应用 reload。

③ [验证] 区分镜像引用、image ID 与当前实例

```
podman image inspect registry.example.com/rhcsa/report:9 \
  --format '{{.Id}}'
podman inspect report --format '{{.Image}} {{.ImageName}}'
```

只有当前实例的 `.Image` 与期望 image ID 一致，才能证明它采用了当前本地镜像。更新后重新验证端口、环境、挂载、数据和功能。自动更新机制属于扩展，不在 RHCSA 主线完整展开。

Cheatsheet

pull 改变本地镜像，不直接替换容器；源变更要 reload；实例采用新配置要 restart/recreate；更新后回归数据和业务功能。

[诊断专题] linger 已启用，但 unit 没有生成

症状可能是： `loginctl show-user` 显示 `Linger=yes`，但 `systemctl --user status report.service` 返回 `unit not found`。此时 `linger` 已经排除一个生命周期问题，但不能证明 `Quadlet` 已被目标 manager 读取。

① [诊断] 先确认作用域和版本

```
id
podman version
systemctl --user --version
loginctl show-user "$USER" -p Linger
```

假设 A：命令在错误用户下执行。下一条最有区分度的证据是 `id` 与 `echo $HOME`，再比较声明的所有者和路径。

假设 B：Podman 版本低于 Quadlet 支持范围。下一条证据是 `podman version` 和本机 `man podman-systemd.unit` 是否存在相关 `.container` 说明。

② [诊断] 检查路径、后缀和内容

```
find ~/.config/containers/systemd -maxdepth 1 -type f -printf '%f\n'
sed -n '1,200p' ~/.config/containers/systemd/report.container
```

常见错误：

- 放到 `~/.config/systemd/user/`；
- 文件名是 `report.service` 或 `report.container.txt`；
- `[Container]` 拼写错误；
- 缺少必需的 `Image=`；
- 文件属于其他用户；
- 编辑器写入不可见字符或错误换行。

③ [诊断] reload 后从 manager 日志取证

```
systemctl --user daemon-reload
systemctl --user list-unit-files | grep -F report
journalctl --user -b --since '-10 min' --no-pager
```

最小修复：只修正错误路径、字段或用户归属，再 `reload`。不要为了让 `unit` 出现而另写一份手工 `.service`，否则会绕过本章声明主线并制造两个配置真源。

再验证：`report.service` 可加载、来源指向 `.container`，然后才进入启动层。

Cheatsheet

``Linger=yes + unit not found`` 优先查用户、版本、路径、后缀、必需字段和 `generator` 日志；不要先改成 `rootful`，也不要另建同名手工 `service`。

[诊断专题] unit 已生成，但容器启动失败或循环重启

这里的目标是从 `systemd` 结果推进到 Podman 和宿主资源，而不是反复执行 `restart`。

① [诊断] 读取最小状态向量

```
systemctl --user show report.service \
  -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
journalctl --user-unit=report.service -b -n 100 --no-pager
```

`Result=exit-code`、`NRestarts` 增长和 `journal` 中首个失败原因，比最后一行“Scheduled restart job”更有价值。

② [诊断] 按错误类型进入下一层

日志线索	假设	下一条证据
name already in use	同名手工实例残留	<code>podman ps -a --filter name=report</code>
address already in use	宿主端口冲突	<code>ss -ltnp grep :18080</code> 、 <code>podman ps --format</code>
permission denied	宿主 DAC、UID 映射或 SELinux	<code>namei -l</code> 、 <code>ls -ldZ</code> 、AVC 日志
image not known/pull failed	镜像引用或网络/认证	<code>podman image exists</code> 、手工 <code>podman pull</code> 的错误
容器很快 Exited	应用入口或参数失败	<code>podman ps -a</code> 、 <code>podman logs</code>

手工运行诊断命令时不要轻易创建另一个同名容器；优先读取 generated service、journal 和失败实例证据。

③ [诊断] 最小修复后清除失败并重验

修正根因后：

```
systemctl --user daemon-reload # 仅当声明有修改
systemctl --user reset-failed report.service
systemctl --user restart report.service
```

再按 unit、容器、功能和数据四层验证。`reset-failed` 只清理失败计数和状态，不修复配置。

Cheatsheet

先看 `Result/NRestarts` 和第一失败原因；重名查容器，端口查监听，权限查 DAC/映射/SELinux，退出查应用日志；修根因后再 restart。

[诊断专题] 当前运行正常，但注销或重启后失败

此类故障说明当前运行层已经通过，问题集中在 user manager 生命周期、安装关系或“未登录验证方法”上。

① [诊断] 分开检查 [Install] 与 linger

源声明：

```
sed -n '/^\[Install\]/,$p' ~/.config/containers/systemd/report.container
```

linger：

```
loginctl show-user appsvc -p Linger
```

两者缺一不可：`WantedBy=default.target` 表达 manager 启动时拉起 service；linger 支持 manager 在开机和注销后存在。

② [诊断] 注销测试不能依赖同一个会话

在目标用户会话中确认当前功能后，退出该用户全部会话。从另一个管理员账号或远端客户端验证：

```
curl -fsS http://HOST:18080/  
loginctl show-user appsvc -p Linger -p State
```

若重新登录目标用户后服务才恢复，说明“登录触发启动”，不能算作未登录持久成功。

③ [诊断] 冷启动证据应先于目标用户登录

重启后不要先登录 appsvc。先从其他账号或外部主机测试业务入口，再查看：

```
loginctl user-status appsvc
```

随后进入目标用户完整会话，核对：

```
systemctl --user status report.service --no-pager  
journalctl --user-unit=report.service -b --no-pager  
podman ps -a
```

本会话没有 RHEL 9 VM，以上属于推荐 live-test 流程，不能声明已执行。

Cheatsheet

当前 active 只证明现在；`[Install]` 解决 manager 内的启动关系，linger 解决 manager 的登录外生命周期；冷启动验收要在目标用户首次登录前先测功能。

[经典任务] 把现有 rootless 容器迁移为 Quadlet，并验证未登

录持久运行

环境

服务器上存在普通用户 `appsvc`。该用户当前手工运行一个名为 `report` 的 rootless 容器：

```
镜像: registry.example.com/rhcsa/report:9
宿主端口: 18080
容器端口: 8080
宿主数据目录: /home/appsvc/report-data
容器数据目录: /var/lib/report
环境变量: REPORT_MODE=exam
```

`/home/appsvc/report-data/marker.txt` 是必须保留的数据。当前容器可以访问，但 `appsvc` 注销后不再可靠运行，系统重启后也未建立未登录启动证据。

目标终态

1. 以 `appsvc` 的 rootless Podman 运行，不改成 rootful。
2. 使用 `~/.config/containers/systemd/report.container` 作为维护真源。
3. 生成并运行 `report.service`，容器名称保持 `report`。
4. 端口、环境变量和数据挂载符合题目。
5. 异常退出时采用 `Restart=on-failure`，间隔 5 秒。
6. 用户注销后服务继续运行。
7. 主机重启后，在 `appsvc` 尚未登录时，外部仍能访问宿主端口。
8. 标记文件在迁移、注销和重启后仍存在。

限制条件

- 不得 `chmod 777`；
- 不得关闭 SELinux；
- 不得删除 `/home/appsvc/report-data`；
- 不得把数据只复制进容器可写层；
- 不得直接编辑 generated service；
- 不得以“重新登录后服务恢复”代替未登录启动验证；
- 删除旧容器前必须确认持久数据位置。

验收证据

层	必须给出的证据
身份	<code>id</code> 、目标用户的 <code>podman ps -a</code>

层	必须给出的证据
声明	<code>.container</code> 路径、关键字段
生成	<code>report.service</code> 可加载且来源正确
unit	<code>ActiveState</code> 、 <code>SubState</code> 、 <code>Result</code> 、 <code>NRestarts</code>
实例	名称、镜像、端口、环境和挂载
数据	宿主和容器内都能读到标记文件
功能	从宿主或远端请求 <code>18080</code> 成功
生命周期	<code>Linger=yes</code> 、注销后功能、冷启动未登录功能
日志	当前 boot 的 user unit journal 无持续失败

[参考解答] 经典任务一

以下命令中的主机名、镜像可访问性和应用返回内容由实际实验环境决定。本章只提供静态核对后的参考流程，不声称已在 RHEL 9 VM 执行。

一、调查当前实例和数据

以 `appsvc` 的完整登录环境操作：

```
id
podman version
podman ps -a --filter name=report
podman inspect report > ~/report-before.json
podman port report
podman inspect report --format '{{json .Mounts}}'
ls -lZ ~/report-data/marker.txt
```

确认 `marker.txt` 位于宿主目录，而不是只存在容器可写层。若当前实例没有正确挂载，应先安全导出数据，不能直接删除实例。

记录当前功能：

```
curl -fsS http://127.0.0.1:18080/
```

二、建立 Quadlet 声明

```
install -d -m 0755 ~/.config/containers/systemd
cat > ~/.config/containers/systemd/report.container <<'EOF'
[Unit]
Description=Persistent rootless report container

[Container]
Image=registry.example.com/rhcsa/report:9
ContainerName=report
PublishPort=18080:8080
Volume=/home/appsvc/report-data:/var/lib/report:Z
Environment=REPORT_MODE=exam

[Service]
Restart=on-failure
RestartSec=5
TimeoutStartSec=900

[Install]
WantedBy=default.target
EOF
```

静态核对：

```
sed -n '1,200p' ~/.config/containers/systemd/report.container
ls -l ~/.config/containers/systemd/report.container
```

`TimeoutStartSec=900` 用于降低首次拉取镜像超过默认超时的风险；若镜像已预拉且环境不需要，可省略。不要通过 `PodmanArgs=` 绕过有明确字段的标准配置。

三、处理同名旧实例

由于声明显式使用 `ContainerName=report`，旧实例若仍存在会发生重名。先停止当前业务并再次确认数据：

```
podman stop report
ls -lZ ~/report-data/marker.txt
```

只有确认数据已经外置后，才删除旧实例：

```
podman rm report
```

若题目不允许删除现有容器，可暂时不指定 `ContainerName=`，接受默认名称，并在维护窗口另行迁移；但这不满足本任务“名称保持 report”的终态。

四、重新生成并启动

```
systemctl --user daemon-reload
systemctl --user start report.service
```

检查 generated unit:

```
systemctl --user status report.service --no-pager
systemctl --user show report.service \
    -p LoadState -p ActiveState -p SubState -p Result -p NRestarts
systemctl --user cat report.service
```

不要对 generated service 机械执行:

```
systemctl --user enable report.service
```

开机关系已经写在 `.container` 的 `[Install]` 中, 由 generator 应用。

五、验证容器参数、功能与数据

```
podman ps -a --filter name=report
podman inspect report --format '{{.Name}} {{.ImageName}} {{.Image}}'
podman port report
podman inspect report --format '{{json .Config.Env}}'
podman inspect report --format '{{json .Mounts}}'
podman exec report cat /var/lib/report/marker.txt
cat ~/report-data/marker.txt
curl -fsS http://127.0.0.1:18080/
```

如果 unit active 但 `curl` 失败, 继续检查应用监听地址、容器内端口、应用日志和主机防火墙; 不要把 active 扩大解释为功能正确。

六、启用并验证 linger

由有权限的管理员执行:

```
sudo loginctl enable-linger appsvc
loginctl show-user appsvc -p Linger
```

预期证据是 `Linger=yes`。这一步不替代 `.container` 和 service 验证。

七、验证注销后运行

退出 `appsvc` 的全部登录会话。使用另一个账号或远端主机验证:


```
curl -fsS http://SERVER:18080/  
loginctl show-user appsvc -p Linger -p State
```

若只有重新登录 `appsvc` 后才恢复，继续调查 `[Install]`、generator 生成关系和 user manager，而不是宣告完成。

八、验证重启后未登录运行

在得到授权和维护窗口后重启。主机起来后，先不要登录 `appsvc`。从另一个账号或远端客户端执行功能测试：

```
curl -fsS http://SERVER:18080/  
loginctl user-status appsvc
```

随后进入 `appsvc` 的完整登录环境，回查：

```
systemctl --user status report.service --no-pager  
systemctl --user show report.service \  
-p ActiveState -p SubState -p Result -p NRestarts  
journalctl --user-unit=report.service -b --no-pager  
podman ps -a --filter name=report  
podman exec report cat /var/lib/report/marker.txt
```

九、典型错误

1. Quadlet 放错目录： `~/.config/systemd/user/report.container` 不属于本章 rootless Quadlet 搜索路径。
2. 只开 `linger`： `manager` 可以存在，但没有正确 generated unit，容器仍不会启动。
3. 只看 `active`： 应用可能返回错误、端口不可达或数据挂载错误。
4. 直接 `enable` generated service： 忽略 Quadlet transient unit 的 `[Install]` 机制。
5. 先删容器再找数据： 若数据在可写层，会造成不可恢复损失。
6. 用 `chmod 777` 修权限： 破坏最小权限，且可能仍绕不过 SELinux 或 UID 映射。
7. 登录后验证冷启动： 登录动作本身可能创建 `manager` 或触发 unit，污染证据。

[经典任务] 诊断 `linger` 已启用但 unit 未生成，并处理重启循环

场景

用户 `websvc` 的 `Linger=yes`。管理员声称已经创建容器服务，但出现两组症状：

1. `systemctl --user status portal.service` 最初返回 `unit not found`；
2. 修改后 unit 出现，却快速进入 `failed`，并且 `NRestarts` 持续增长。

已知系统中存在：

- 文件 `~/.config/systemd/user/portal.container`；
- 一个停止状态的手工容器 `portal`；
- 宿主端口 `18090` 可能被其他进程使用；
- 数据目录 `~/portal-data` 不得删除；
- 禁止改成 `rootful`、禁止关闭 SELinux、禁止用 `Restart=always` 掩盖错误。

要求

按“症状 -> 当前证据 -> 假设 -> 下一条区分度最高的证据 -> 最小修复 -> 再验证”提交处理过程，并最终满足：

- `portal.container` 位于正确路径；
- generated `portal.service` 可加载；
- 容器名称和端口无冲突；
- 数据目录可由应用访问；
- unit 不再循环；
- 协议功能和持久数据通过验证。

[参考解答] 经典任务二

第一阶段：unit not found

症状： `portal.service` 不存在。

当前证据：

```
id
loginctl show-user webservice -p Linger
find ~/.config -maxdepth 4 -type f -name 'portal*' -print
```

假设： `.container` 放入普通 user unit 目录，而不是 Quadlet 目录。

下一条最有区分度的证据： 路径显示为：

```
/home/webservice/.config/systemd/user/portal.container
```

最小修复：

```
install -d -m 0755 ~/.config/containers/systemd
mv ~/.config/systemd/user/portal.container \
  ~/.config/containers/systemd/portal.container
systemctl --user daemon-reload
```

再验证：

```
systemctl --user status portal.service --no-pager
systemctl --user show portal.service -p LoadState -p SourcePath -p FragmentPath
```

若仍不存在，继续查 Podman 版本、`Image=`、文件后缀和 user journal，而不是创建同名手工 service。

第二阶段：restart loop

症状：service 出现但快速失败。

当前证据：

```
systemctl --user show portal.service \
  -p ActiveState -p SubState -p Result -p NRestarts
journalctl --user-unit=portal.service -b -n 100 --no-pager
```

假设按日志分支：

分支 A：容器重名

```
podman ps -a --filter name=portal
```

若旧手工实例存在，先确认其挂载和数据：

```
podman inspect portal --format '{{json .Mounts}}'
ls -lZ ~/portal-data
```

确认数据外置后，停止并删除旧实例：

```
podman stop portal 2>/dev/null || true
podman rm portal
```

分支 B：宿主端口冲突

```
ss -ltnp | grep ':18090 '
podman ps --format 'table {{.Names}}\t{{.Ports}}'
```

如果题目指定端口，停止或调整真正冲突对象前先确认业务归属；不能随意改成随机端口并宣告完成。

分支 C：数据目录访问失败

```
namei -l ~/portal-data
ls -ldZ ~/portal-data
podman inspect portal --format '{{json .Config.User}}' 2>/dev/null
```

先修正最小传统权限和所有者，再检查 AVC。不得 `chmod 777`，不得 `setenforce 0`。

第三阶段：再生成、再启动和分层验证

声明有修改时：

```
systemctl --user daemon-reload
systemctl --user reset-failed portal.service
systemctl --user restart portal.service
```

验证：

```
systemctl --user show portal.service \
  -p ActiveState -p SubState -p Result -p NRestarts
podman ps -a --filter name=portal
podman port portal
podman exec portal test -r /var/lib/portal/marker.txt
curl -fsS http://127.0.0.1:18090/
journalctl --user-unit=portal.service -b -n 50 --no-pager
```

`NRestarts` 在一次历史故障后不一定立即变成零；关键是修复后不再持续增长，当前 `Result` 和业务功能正确。

[本章收束] 把容器持久性拆成一条可执行的工作方法

面对“让 rootless 容器重启后自动运行”这类要求，不要直接从模板或启动命令开始。先把任务翻译为七个可证明的对象：目标 UID、现有实例、源声明、generated unit、运行实例、外部数据和未登录功能。任何一层没有证据，都不能用下一层的成功代替。

1 确认身份与版本目标用户、rootless 存储、Podman 版本、本机 man page

2 保存当前基线实例参数、端口、挂载、数据位置和当前功能

3 建立源声明把题目终态翻译为 Quadlet 字段和 `[Install]`

4 重新生成与切换 reload 生成 unit，安全处理旧实例，再 start/restart

5 分层验收 manager、实例、数据、协议和 journal 分别取证

6 验证登录外生命周期 linger、注销测试、冷启动未登录测试

章末检查清单

- [] 操作命令连接的是目标用户的 user manager，而不是 system scope。
- [] podman version 和本机 podman-systemd.unit(5) 支持所用 Quadlet 语义。
- [] 现有实例的镜像、端口、环境、挂载和数据位置已保存为基线。
- [] .container 位于正确搜索路径，文件名、后缀和所有者正确。
- [] Image、ContainerName、PublishPort、Volume、Environment 与目标终态一致。
- [] [Install] WantedBy=default.target 与 Linger=yes 分别完成自己的职责。
- [] reload、restart 的先后和作用对象清楚，未直接编辑 generated unit。
- [] unit、容器、数据、协议功能和日志都已分别验证。
- [] 持久数据经过实例删除和重建验证，而不仅是简单 restart。
- [] 冷启动功能测试发生在目标用户首次登录之前。

主要判断表

看到的证据	能证明什么	仍不能证明什么	下一条高区分度证据
.container 文件存在	源声明文件已放置	generator 已识别、字段正确	daemon-reload 后检查 LoadState 和 user journal
report.service 可加载	generated unit 已出现	unit 能启动、实例参数正确	systemctl --user show 与 journal
ActiveState=active	manager 当前认为服务活动	应用响应、数据挂载、冷启动成功	podman inspect、协议请求、数据读取
podman ps 显示运行	容器进程存在	对外功能正确、数据独立	podman port、curl、宿主与容器内文件对照
Linger=yes	user manager 可脱离登录会话存在	Quadlet 会启动、业务可用	注销后和冷启动未登录的外部功能测试
restart 后文件仍在	当前重启未丢失文件	删除和重建后仍持久	删除实例并由声明重建，再读取外部数据
journal 无新错误	当前查询范围内未见失败记录	业务终态完全正确	协议、内容和持久数据验收

向下一章交接

本章已经把单个容器转换为可维护、可重建、可诊断的用户级服务。进入下一章“RHCSA 综合任务与证据矩阵”时，不再重复 Quadlet 字段，而是把这里的证据链与用户、网络、存储、SELinux、防火墙和重启验收组合成整机终态。

静态可信边界：本章依据 RH134、RHEL 9 容器文档、Podman 4.6.1 Quadlet 手册和 systemd 手册进行静态核对。当前会话没有可控 RHEL 9 VM，未执行真实注销、冷启动、端口请求、实例删除与重建测试；这些项目保留在 revision-notes.md 的 live-test 清单中。